



C++ Programming Language

Presented By

Usha Rani Sargurunathan

STG Group

Operator Overloading

Operator Overloading

Operator overloading allows you to **redefine** how C++ operators (like `+`, `-`, `==`, `++`) work with your own user-defined types. This makes code more intuitive and readable.

- **Overloading unary and binary operators:** You can overload most operators as a member function or a non-member function.
- **Overloading the `>>` and `<<` operators:** These are typically overloaded as non-member **friend** functions to allow them to access the private data of a class. This makes it possible to use standard input/output streams with your custom objects.
- **Operators that cannot be overloaded:** A few operators cannot be overloaded, including the scope resolution operator (`::`), member access operator (`.`), pointer-to-member operator (`.*`), and the ternary conditional operator (`?:`).

Example-1

```
#include <iostream>
using namespace std;
int main()
{
    int a=6, b=10;
    cout<< a+b<<endl;
    string str1="Hello", str2="World";
    cout<< str1+str2<<endl;
}
```

Example-2

```
#include <iostream>
using namespace std;
class Time {
private:
    int hour;
    int minute;
public:
    Time(int h, int m) {
        hour = h;
        minute = m; }
    void addminute() {
        minute++;
        if(minute >= 60) {
            hour++;
            minute -= 60; } }
```

```
void displayTime()
{
    cout << "Time: " << hour << ":" <<
    minute << endl;
};

int main()
{
    Time t(10, 30);
    t.addminute();
    t.displayTime();
}
```

Example 2 – Overload ++ operator(pre)

```
#include <iostream>
using namespace std;
class Time {
private:
    int hour;
    int minute;
public:
    Time(int h, int m) {
        hour = h;
        minute = m; }
    void operator++() {
        minute++;
        if(minute >= 60) {
            hour++;
            minute -= 60; } }
```

```
void displayTime()
{
    cout << "Time: " << hour << ":" <<
    minute << endl;
};

int main()
{
    Time t(10, 30);
    ++t; // t.operator++() compiler writes it
    t.displayTime();
}
```

Key Rules for Overloading ++ in C++

Prefix (++t): Takes no dummy parameters:

```
void operator++() { ... }
```

Postfix (t++): Requires an int parameter as a dummy argument to tell the compiler it is postfix:

```
void operator++(int) { ... }
```

Syntax of Operator Overloading:

```
return_type operator symbol(parameters) {  
    // function body  
}
```

Example 2 – Overload ++ operator(post)

```
#include <iostream>
using namespace std;
class Time {
private:
    int hour;
    int minute;
public:
    Time(int h, int m) {
        hour = h;
        minute = m; }
    void operator++() {
        minute++;
        if(minute >= 60) {
            hour++;
            minute -= 60; } }
```

```
void operator++(int) {
    minute++;
    if(minute >= 60) {
        hour++;
        minute -= 60; } }
void displayTime()
{
    cout << "Time: " << hour << ":" <<
    minute << endl;
}
};
int main() {
    Time t(10, 30);
    ++t; // t.operator++() compiler writes it
    t++; // t.operator++(0) compiler writes it
    t.displayTime(); }
```

List of Operators that can be Overloaded

+	-	*	/	%	^
&		~	!	,	=
<	>	<=	>=	++	--
<<	>>	==	!=	&&	
+=	-=	/=	%=	^=	&=
=	*=	<<=	>>=	[]	()
->	->*	new	new []	delete	delete []

Operators that cannot be used overloaded

::	.*	.	?:
----	----	---	----

Rules of Operator Overloading

1. Precedence and associativity of operator cannot be changed.
2. Only existing operators can be overloaded.
3. Meaning of how the operator works should not be changed.
4. Return types should be compatible with built in types.

Example-3 Overload Input Output Operator (<<, >>)

```
#include <iostream>
using namespace std;
class Room{
    private:
        int length;
        int breadth;
};
int main() {
    Room r1;
    cin>>r1;
}
```

```
#include <iostream>
using namespace std;
class Room{
    private:
        int length;
        int breadth;
    public:
        friend istream& operator>>(istream &input, Room &r);
        friend ostream& operator<<(ostream &output, Room &r){
            output<<r.length<<"X"<<r.breadth<<endl;
            return output; } };
    istream& operator>>(istream &input, Room &r){
        cout<<"Enter length and breadth of room: ";
        input>>r.length>>r.breadth;
        return input; }
int main(){
    Room r1;
    cin>>r1;
    cout<<r1; }
```

Task

Define a Date class with day, month and year as data.

Get input date by overloading >> operator.

Print the date by overloading << operator.

Solution

```
#include <iostream>
using namespace std;
class Date{
private:
    int date;
    int month;
    int year;
public:
    friend istream& operator>>(istream &input, Date &d);
    friend ostream& operator<<(ostream &output, Date &d){
        output<<d.date<<"/"<<d.month<<"/"<<d.year<<endl;
        return output; } };
```

```
istream& operator>>(istream &input,
Date &d) {
    cout<<"Enter date, month
and year: ";
    input>>d.date>>d.month>>d.year;
    return input; }
int main()
{
    Date d1;
    cin>>d1;
    cout<<d1;
}
```

Output:

```
Enter date, month and year: 1 9 2026
1/9/2026
```

Example-4 Binary Operator(+) Overloading

```
#include <iostream>
using namespace std;
class Distance {
private:
    int feet;
    int inches;
public:
    friend istream& operator>>(istream &input, Distance &d);
    friend ostream& operator<<(ostream &output, Distance &d);
    Distance operator+(Distance d) {
        Distance temp;
        temp.feet = this->feet + d.feet; //this->feet is d1.feet, d.feet is d2.feet
        temp.inches = this->inches + d.inches;
        if(temp.inches > 11) {
            temp.feet ++;
            temp.inches -= 12; }
        return temp; }    };
```

```
istream& operator>>(istream &input,
Distance &d)
{
    cout<<"Enter feet and inches(0-11): ";
    input>>d.feet>>d.inches;
    return input;
}
ostream& operator<<(ostream &output,
Distance &d)
{
    output<<d.feet<<" feet "<<d.inches<<"
inches"<<endl;
    return output;
}
int main() {
    Distance d1, d2, d3;
    cin>>d1;
    cin>>d2;
    d3 = d1 + d2; //d3 = d1.operator+(d2);
    cout<<"Sum of distances: "<<d3;
    return 0;
}
```

Example-4 Binary Operator(<) Overloading

```
#include <iostream>
using namespace std;
class Distance {
private:
    int feet;
    int inches;
public:
    friend istream& operator>>(istream &input, Distance &d);
    friend ostream& operator<<(ostream &output, Distance &d);
    bool operator<(Distance d) {
        if(this->feet < d.feet)
            return true;
        if(this->feet == d.feet && this->inches < d.inches)
            return true;
        return false;
    };
};
```

```
int main() {
    Distance d1, d2, d3;
    cin>>d1;
    cin>>d2;
    if (d1<d2) {
        cout << "d1 is shorter" << endl;
    } else {
        cout << "d1 is larger" << endl;
    }
    return 0;
}
```

Example – 5

```
#include <iostream>
class Vector {
private:
    int x, y;
public:
    Vector(int x_val = 0, int y_val = 0) : x(x_val),
y(y_val) {}
    // Overload the binary + operator as a member
function
    Vector operator+(const Vector& other) const {
        return Vector(x + other.x, y + other.y);
    }

    // Overload the unary ++ operator (prefix) as a
member function
    Vector& operator++() {
        ++x;
        ++y;
        return *this;
    }
    // Friend function to overload the << stream
insertion operator
    friend std::ostream& operator<<(std::ostream&
os, const Vector& v);
};
```

```
// Definition of the overloaded << operator
std::ostream& operator<<(std::ostream& os, const
Vector& v) {
    os << "Vector(" << v.x << ", " << v.y << ")";
    return os; }

int main() {
    Vector v1(1, 2);
    Vector v2(3, 4);
    Vector v3 = v1 + v2; // Calls the overloaded +
operator

    std::cout << "v1: " << v1 << std::endl;
    std::cout << "v2: " << v2 << std::endl;
    std::cout << "v1 + v2 = " << v3 << std::endl;
    ++v1; // Calls the overloaded prefix ++ operator
    std::cout << "After ++v1: " << v1 << std::endl;
    return 0;
}
```

```
v1: Vector(1, 2)
v2: Vector(3, 4)
v1 + v2 = Vector(4, 6)
After ++v1: Vector(2, 3)
```

Inheritance and Polymorphism

The Inheritance Mechanism

- **Definition:** A fundamental OOP mechanism that allows a new class (the **derived class** or **subclass**) to inherit properties (data members) and behaviors (member functions) from an existing class (the **base class** or **superclass**).
- **Purpose:**
 - **Code Reusability:** Avoids duplicating code by allowing derived classes to reuse the implementation of base classes.
 - **Extensibility:** Allows new functionalities to be added to existing classes without modifying them.
 - **Polymorphism:** Enables treating derived class objects as base class objects, facilitating flexible and generic code.
 - **Modeling "Is-A" Relationships:** Represents a specialization or subtype relationship (e.g., a *Car is a Vehicle*).
- **Syntax:** `class DerivedClass : access_specifier BaseClass { /* ... */ };`
 - **access_specifier:** `public`, `protected`, or `private`.

The Inheritance Mechanism

Inheritance establishes a parent-child relationship between classes. The derived class inherits the members of the base class and can add its own new members or override inherited ones. This promotes **code reusability** and a hierarchical structure.

Types of Inheritance

- **Single:** A derived class inherits from a single base class.
- **Multi-level:** A class inherits from a derived class (e.g., **Class C** inherits from **Class B**, which inherits from **Class A**).
- **Multiple:** A derived class inherits from multiple base classes.
- **Hierarchical:** Multiple classes inherit from a single base class.
- **Hybrid:** A combination of two or more types of inheritance.

Real-time Use Cases

Base

Student

Employee

Account

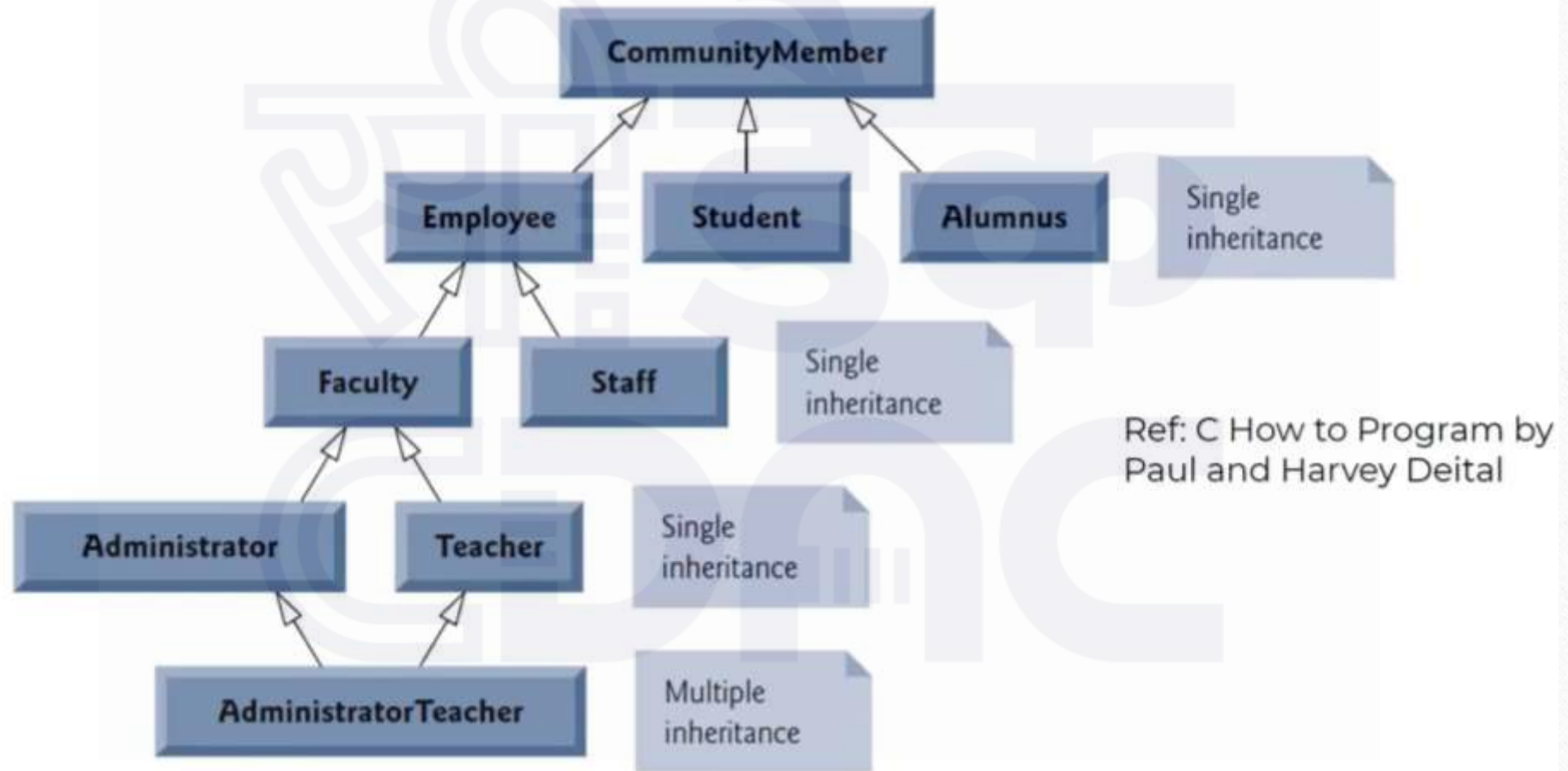
Derived

UnderGraduate, PostGraduate

FullTime, Contract

SavingsAccount, CurrentAccount

Example Scenario



Types of Inheritance

- **Single Inheritance:** A class inherits from only one base class.
 - `class Dog : public Animal { /* ... */ };`
- **Multi-level Inheritance:** A derived class inherits from a base class, which itself is a derived class of another base class.
 - `class Animal { /* ... */ };`
 - `class Mammal : public Animal { /* ... */ };`
 - `class Dog : public Mammal { /* ... */ };`
- **Multiple Inheritance:** A class inherits from multiple base classes.
 - `class Bat : public Mammal, public Flyer { /* ... */ };`
- **Hierarchical Inheritance:** Multiple classes inherit from a single base class.
 - `class Animal { /* ... */ };`
 - `class Dog : public Animal { /* ... */ };`
 - `class Cat : public Animal { /* ... */ };`
- **Hybrid Inheritance:** A combination of two or more types of inheritance (e.g., multiple and hierarchical). Often involves virtual base classes to resolve ambiguity.

Relationship

- Class inheriting from another class – Is-A relationship
- Class has an object of another class – Has-A relationship

Example:

```
Class Rectangle{
```

```
-----
```

```
}
```

```
Class Cuboid : public Rectangle {
```

```
-----
```

```
}
```

```
Class Table{
```

```
Rectangle top; // has an object of rectangle class
```

```
int legs;
```

```
}
```

Here Table class is not inherited

Example 1

```
#include <iostream>
using namespace std;
class Base {
public:
    int a;
    void display() {
        cout<<"Display of Base " << a
        << endl; }
};
```

```
class Derived:public Base {
public:
    void show() {
        cout<<"Show of Derived"<<endl; }
};
int main() {
    Derived d;
    d.a=100;
    d.display();
    d.show(); }
```

Example 2

```
#include <iostream>
using namespace std;
class Base {
public:
    Base() {
        cout<<"Non-param Base"
        << endl; }
    Base(int x) {
        cout<<"Param of Base" <<
        x << endl; }
};
```

```
class Derived:public Base{
public:
    Derived() {
        cout<<"Non-Param Derived"<<endl;}
    Derived(int y) {
        cout<<"Param of Derived " <<y<<endl;}
    Derived(int x,int y):Base(x) {
        cout<<"Param of Derived " << y <<
        endl; }
};
int main() {
    Derived d(5,10); }
```

Summary Table

- A **derived class constructor** always calls a **base class constructor** first.
- If you **do not specify** a base class constructor, the compiler calls the **default (non-parameterized) constructor**.
- To call a **parameterized base class constructor**, use the **initializer list**:

Derived(int x, int y) : Base(x)

- The base class constructor finishes execution **before** the derived class constructor body starts.

Object Created	Base Constructor Called	Derived Constructor Called	Output
Derived d;	Base()	Derived()	Non-param Base → Non-Param Derived
Derived d(10);	Base()	Derived(int)	Non-param Base → Param of Derived 10
Derived d(5,10);	Base(5)	Derived(int,int)	Param of Base 5 → Param of Derived 10

Important Rule

Whenever a **derived class object** is created:

1. The **base class constructor** executes first.
2. Then the **derived class constructor** executes.

Similarly, during **object destruction**:

1. The **derived class destructor** executes first.
2. Then the **base class destructor** executes.

```

#include<iostream>
using namespace std;
class Employee {
    int empld;
    string name;
    double salary;
public:
    Employee(int eid, string n, double sal=0.0) :
    empld(eid), name(n), salary(sal) {
    void display() {
        cout << "Employee ID: " << empld << endl;
        cout << "Name: " << name << endl;
        cout << "Salary: $" << salary << endl; }
    void setName(string n) {
        name = n; }
    string getName() {
        return name; }
    void setSalary(double sal) {
        salary = sal; }
    double getSalary() {
        return salary; } };

```

```

class ContractEmployee : public Employee {
    int WorkHours; // in months
public:
    ContractEmployee(int eid, string n, int wh) :
    Employee(eid,n) {
        WorkHours = wh;
        setSalary(); }
    void setSalary() {
        Employee::setSalary(WorkHours * 100.0); //
        Assuming $100 per hour }
    void display() {
        Employee::display();
        cout << "Work Hours: " << WorkHours << endl; } };

int main() {
    Employee e1(101, "Raj", 50000);
    e1.display();
    cout << endl;
    e1.setSalary(20000); // Update salary
    e1.display();
    cout << endl;

    ContractEmployee ce2(102, "Riya", 160); // 160 hours
    ce2.display();
    cout << endl; }

```

Protected Access Specifier

- The protected access specifier in C++ acts as a middle ground between public and private. It hides data from the outside world while explicitly sharing it with child classes.
- **Definition:** A member declared as protected in a base class.
- **Access Rules:**
 - **Within the same class:** Accessible.
 - **Within derived classes:** Accessible (both directly and through objects of the derived class itself).
 - **From outside the class/hierarchy:** Not accessible (behaves like private).
- **Purpose:** To provide a level of access that is intermediate between private (only accessible within the class) and public (accessible everywhere). It allows derived classes to interact with specific base class members without exposing them to the entire program.

Example

```
#include <iostream>
using namespace std;

class Parent {
protected:
    int protectedValue; // Child classes can see this, but the outside world cannot
};

class Child : public Parent {
public:
    void setVal(int x) {
        protectedValue = x; // Allowed: Child class has access
    }
    void display() {
        cout << "Value: " << protectedValue << endl;    };
};

int main() {
    Child obj;
    obj.setVal(42);
    obj.display();
    // obj.protectedValue = 100;    // Compilation Error: 'protectedValue' is protected within this context
    return 0;}
```

Inheritance Types

Base class member access specifier	Public inheritance	Private inheritance	Protected inheritance
Public	Public	Private	Protected
Protected	Protected	Private	Protected
Private	Inaccessible	Inaccessible	Inaccessible

Guess the Output?

```
#include <iostream>

class Base {
private: int a;
protected: int b;
public: int c;
void funBase() {
a=10;
b=5;
c=15; }
};
```

```
class Derived : public Base {
public:
void funDerived() {
a=10;
b=5;
c=15; }
};

int main() {
Base b;
b.a=10;
b.b=5;
b.c=20; }
```

Virtual Function

- **Definition:** A member function declared with the **virtual** keyword in a **base class**. It signals that this **function can be overridden by derived** classes, and its specific implementation should be determined at runtime.
- **Purpose:** To enable **runtime polymorphism** (dynamic binding). When a virtual function is called through a pointer or reference to a base class object, the version of the function in the *actual type* of the object (derived class) is executed.
- **Syntax:** **virtual returnType functionName(parameters);**

Example

```
#include<iostream>
using namespace std;
class Shape{
public:
    double base, height;
    Shape(double b, double h){
        base = b;
        height = h; }
    virtual double area(){
        cout << "Base class area function called"
        << endl;
        return 0; } };
class Triangle: public Shape{
public:
    Triangle(double b, double h): Shape(b,h) {}
    double area(){
        cout << "Derived Triangle class area
function called" << endl;
        return 0.5 * base * height; } };
```

```
class Rectangle: public Shape{
public:
    Rectangle(double b, double h):Shape(b,h){}
    double area(){
        cout << "Derived Rectangle class area
function called" << endl;
        return base * height; } };
int main(){
    Shape *s1;
    Triangle t(10.0, 5.0);
    cout << "Area of triangle: " << t.area() << endl;
    s1 = &t;
    cout << "Area of triangle: " << s1->area() << endl;
    Rectangle r(10.0, 5.0);
    s1 = &r;
    cout << "Area of rectangle: " << s1->area() <<
endl;
    return 0; }
```

Abstract Class

- A pure virtual function is a virtual member function that belongs to a base class and needs to be redefined in a derived classes.
- When a class has a pure virtual function as a member, it becomes an abstract base class. Abstract base classes can never be instantiated.

-> Base classes that make the code reusable and extendable

-> Must have atleast one pure virtual function

-> Cannot create objects for abstract classes

-> Derived class must provide definition for the pure virtual function otherwise it also becomes an abstract class

Pure Virtual Function

- **Definition:** A virtual function in a base class that has no implementation in the base class itself. It is declared by assigning 0 to it.
- **Syntax:** `virtual returnType functionName(parameters) = 0;`
- **Purpose:**
 - To declare an **abstract class** (a class that cannot be instantiated directly).
 - To force derived classes to provide their own implementation for that function, ensuring a common interface for a group of related classes.
- **Characteristics:**
 - A class containing at least one pure virtual function is an **abstract class**.
 - Derived classes *must* implement all inherited pure virtual functions to become concrete (instantiable).

Example – Abstract Class

```
#include<iostream>
using namespace std;
class Shape{
    public:
        double base, height;
        Shape(double b, double h){
            base = b;
            height = h; }
        virtual double area()=0; // pure virtual function
};
class Triangle: public Shape{
    public:
        Triangle(double b, double h):Shape(b,h){}
        double area(){
            cout << "Derived Triangle class area function called" << endl;
            return 0.5 * base * height; }
};
```

```
int main() {
    Shape s;
```

// This line will cause a compilation error because Shape is an abstract class and cannot be instantiated.

```
}
```

Example – Pure Virtual Function

```
#include <iostream>
using namespace std;
class Base
{
public:
virtual void fun1()=0;
virtual void fun2()=0;
};
```

```
class Derived :public Base {
public:
void fun1() {
cout<<"fun1 of Derived"<<endl; }
void fun2() {
cout<<"fun2 of Derived"<<endl; }
};
int main() {
Derived d;
d.fun1();
d.fun2(); }
```

Key Takeaway

- Class having pure virtual function is an abstract class.
- Object of abstract class cannot be created.
- Derived class can must override pure virtual function, otherwise it will also become an abstract class.
- Pointer of abstract class can be created.
- Pointer of abstract class can hold object of derived class.
- Abstract classes are used for achieving polymorphism.
- Abstract class is for extendibility.

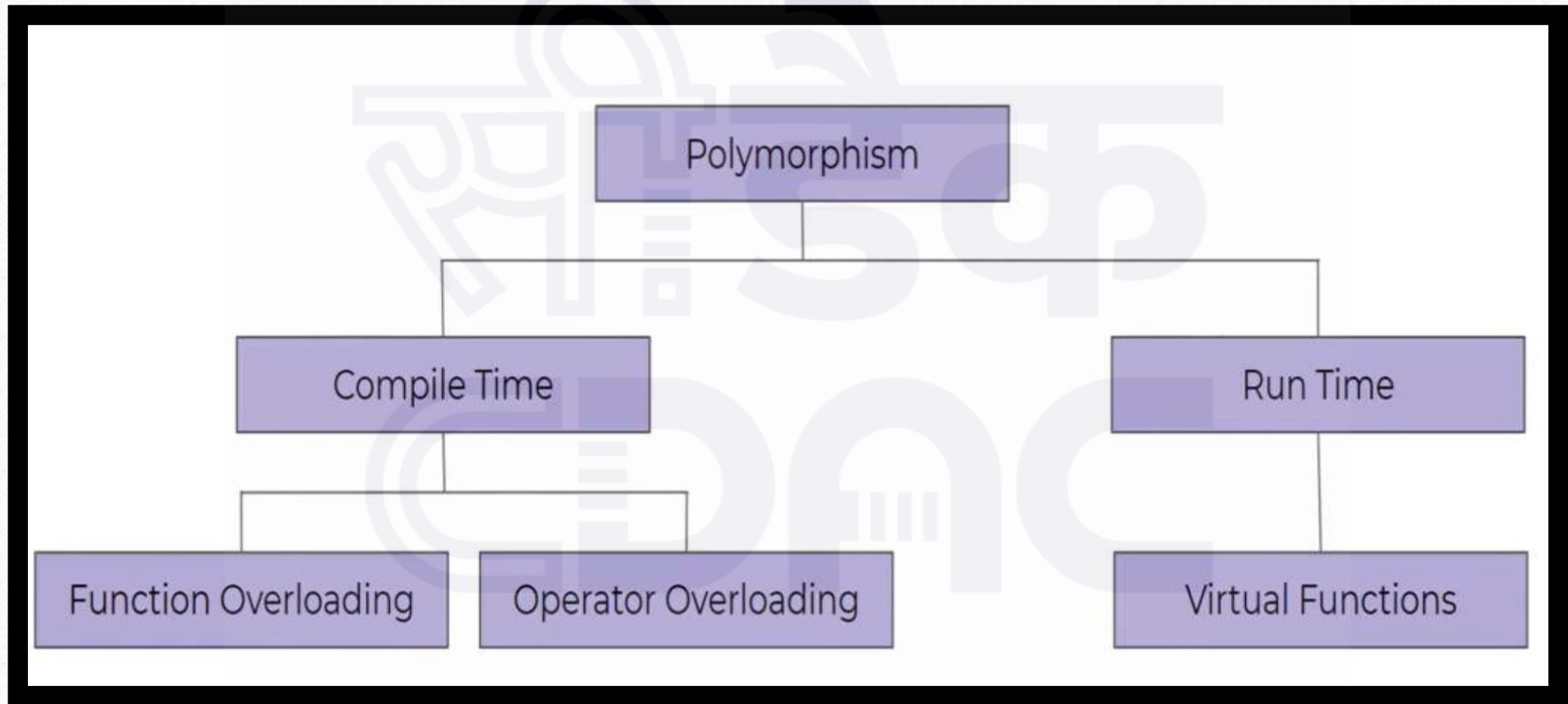
Runtime Polymorphism & Virtual Functions

Runtime Polymorphism & Virtual Functions

Polymorphism (Greek for "many forms") allows an object to take on different forms. **Runtime polymorphism** in C++ is achieved through **virtual functions** and pointers. When you call a **virtual** function through a base class pointer, the program decides which version of the function (base or derived) to execute at runtime based on the actual object type, not the pointer type. This is known as **dynamic binding**.

- A **virtual function** is a member function in the base class that you expect to be overridden by a derived class.
- A **pure virtual function** is a virtual function with no implementation (`= 0`). A class with one or more pure virtual functions is an **abstract class**. You cannot create objects of an abstract class.
- An **interface** is a class with only pure virtual functions. A **concrete class** is a class with no pure virtual functions and can be instantiated.

Polymorphism



Comparison

Feature	Static Binding	Dynamic Binding
When it happens	Compile time (Early binding)	Runtime (Late binding)
How it decides	Based on the variable's declared reference type	Based on the actual object type in memory
Core OOP concept	Method Overloading	Method Overriding (Polymorphism)
Speed	Faster performance	Slightly slower due to runtime lookup
Flexibility	High rigidity; cannot change at execution	High flexibility; adapts to runtime data changes

Task - Virtual Member Function

- Define an *Animal* base class that contains a virtual void `makeSound()` function.
- Implement derived classes *Dog* and *Cat*, overriding `makeSound()` to print "barks" and "meows" respectively.
- Demonstrate runtime polymorphism by passing these derived objects by reference to a function accepting a base class reference.

Solution

```
#include <iostream>
class Animal {
public:
    virtual void makeSound() { // Virtual function
        std::cout << "Animal makes a sound\n"; }
};
class Dog : public Animal {
public:
    void makeSound() override { // Overrides base class virtual function
        std::cout << "Dog barks\n"; } };
class Cat : public Animal {
public:
    void makeSound() override { // Overrides base class virtual function
        std::cout << "Cat meows\n"; } };
int main() {
    Animal* myAnimal1 = new Dog();
    Animal* myAnimal2 = new Cat();
    myAnimal1->makeSound(); // Calls Dog::makeSound() at runtime
    myAnimal2->makeSound(); // Calls Cat::makeSound() at runtime
    delete myAnimal1;
    delete myAnimal2;
    return 0; }
```

Exception Handling

What is Exception?

An **exception** is an event that occurs during the execution of a program that disrupts the normal flow of instructions. It's a way for a program to signal that an unusual or erroneous condition has occurred, which the program may not be able to handle in its usual course of execution.

Here's a breakdown of what that means:

- **Runtime Error:** An exception is a type of runtime error, meaning it happens while the program is running, not while it's being compiled.
- **Disruption:** When an exception occurs, the program stops what it's doing and looks for a way to handle the problem. If it can't find a way, the program will often terminate or crash.

Exception Handling Introduction

Concept: A mechanism in C++ to deal with runtime errors or unusual conditions that disrupt the normal flow of program execution.

Keywords: try, throw, catch.

Mechanism:

Code that might throw an exception is placed in a try block.

If an error occurs, an exception is thrown.

The throw statement transfers control to a catch block that can handle the specific type of exception.

If no matching catch block is found, the program terminates.

Example – Exception Handling

```
#include<iostream>
using namespace std;
int main()
{
    int a,b;
    cout<<"Enter two
numbers: ";
    cin>>a>>b;
    cout<<a/b<<endl;
    cout<<"End of program";
}
```

```
#include<iostream>
using namespace std;
int main() {
    int a,b;
    cout<<"Enter two numbers: ";
    cin>>a>>b;
    try{
        if(b==0)
            throw "Division by zero error";
        cout<<a/b<<endl;
    }
    catch(const char* ch) {
        cout<<"Error occurred: "<<ch<<endl;
    }
    cout<<"End of program"; }
```

```
int main() {
    int a,b;
    cout<<"Enter two
numbers: ";
    cin>>a>>b;
    try{
        if(b==0)
            throw 5;
        cout<<a/b<<endl;
    }
    catch(int i) {
        cout<<"Math Error... "
    }
}
```

Catch Block

Multiple catch block can be given to handle different exceptions.

Default catch block:

```
catch(...)  
{  
    cout<<"Unknown Error..."<<endl;  
}
```

Example – try & catch

```
#include<iostream>
using namespace std;
int main() {
    try{
        int *arr = new int[1000000000000]; // Attempt to allocate a large array
    }
    catch(exception &e) {
        cerr<<"Standard exception: "<<e.what()<<endl;
    }
    cout<<"End of program";
}
```

Order of catch Blocks

- **Issue:** The order of `catch` blocks is crucial. If a more general `catch` block (e.g., `catch (const std::exception& e)`) is placed before a more specific one (e.g., `catch (const std::runtime_error& e)`), the specific block will never be reached, as the general one will catch all derived exceptions first.
- **Rule:** Always place `catch` blocks for derived (more specific) exception types *before* `catch` blocks for base (more general) exception types.

Example of Incorrect Order:

```
// try {  
//     throw std::runtime_error("Specific error");  
// } catch (const std::exception& e) { // Catches first!  
//     std::cerr << "Caught general exception: " << e.what() << std::endl;  
// } catch (const std::runtime_error& e) { // NEVER REACHED  
//     std::cerr << "Caught specific runtime error: " << e.what() << std::endl;  
// }
```

Best Practice Hierarchy

```
try {  
    // Dangerous code here  
}  
catch (const std::runtime_error& e) {  
    // 1. Handles ONLY runtime errors specifically  
}  
catch (const std::exception& e) {  
    // 2. Handles any OTHER standard library exception  
}  
catch (...) {  
    // 3. Handles absolutely anything else (unrecognized types)  
}
```

Task

Write a program that asks for a student's name and age, and uses **exception handling** (using try, catch, and throw) to check if the age is valid.